

Prototype Schematic - Simple Explanation and Review

A plain-English guide to what the circuit is trying to do, what the main parts mean, and what should be checked before building it.

Plain-English summary

This schematic is for a timed red/blue LED lighting system. A 12V power supply feeds two LED current drivers. The Arduino Nano ESP32 acts as the brain, the RTC clock keeps time, and two MOSFETs act as electronic switches for the red and blue LEDs.

Important note

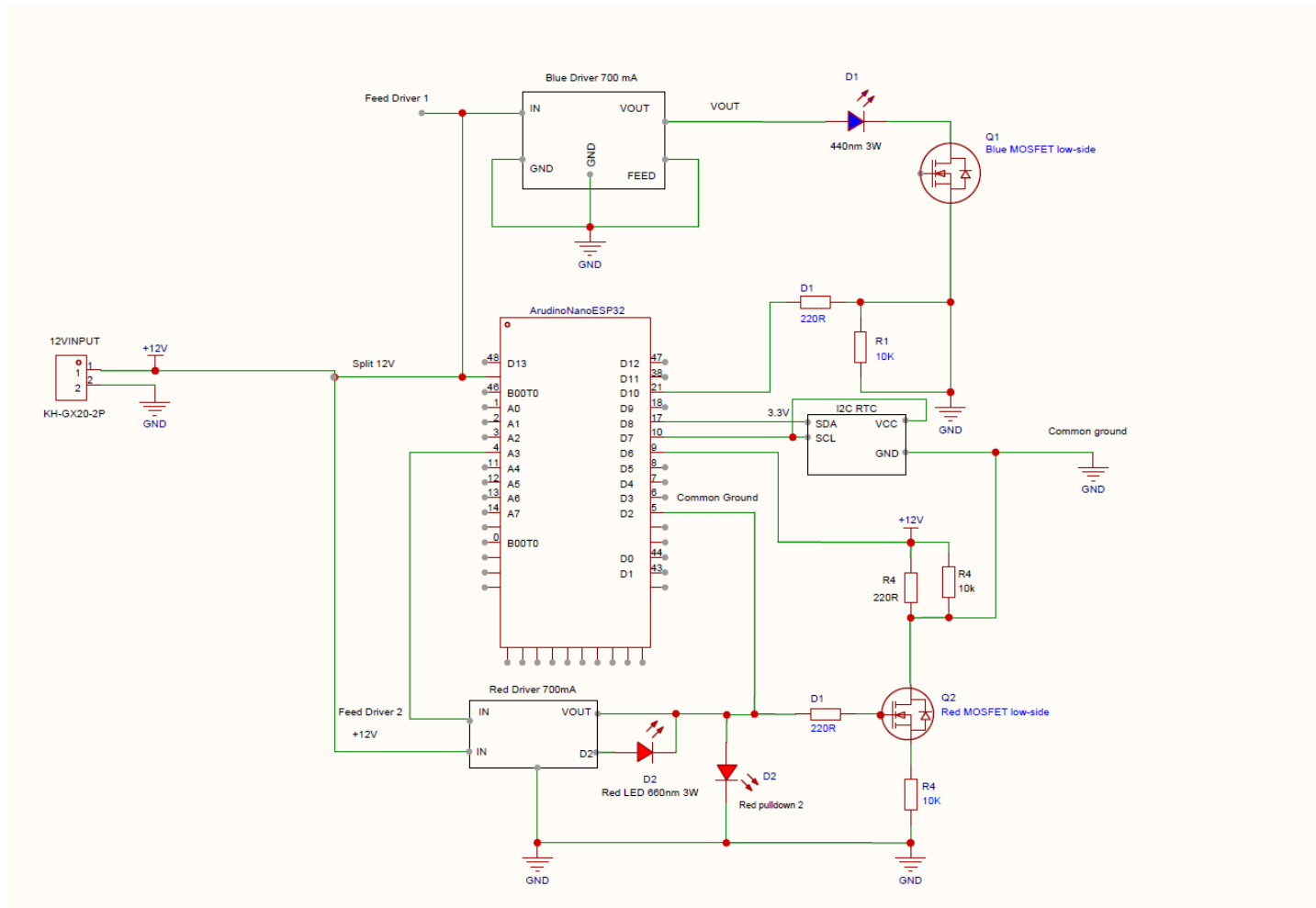
The concept is understandable, but the drawing should be treated as a prototype. Some connections still need to be confirmed against the actual component datasheets before hardware is powered.

What the circuit is trying to do

Step	What happens	Simple meaning
1	12V DC power enters the circuit.	This is the main power source for the high-power LEDs and LED drivers.
2	Each LED has a 700 mA current driver.	The driver protects the LED by controlling the current. The LED should not be powered directly from 12V.
3	The Arduino Nano ESP32 sends control signals.	The Arduino decides when the blue and red LEDs should turn on or off.
4	MOSFETs switch the LED paths.	A MOSFET is like an electronic on/off switch controlled by the Arduino.
5	The RTC module provides time.	The clock lets the Arduino run the LEDs on a schedule.
6	All grounds are tied together.	The Arduino, power supply, RTC, drivers, and MOSFETs need a shared reference called common ground.

Schematic image being reviewed

This image shows the current prototype drawing. It is useful for explaining the system, but the checklist on the next page should be completed before building the physical circuit.



Reading tip: follow the power path from left to right: 12V input -> driver -> LED -> MOSFET switch -> ground. Then follow the control path from the Arduino pins to the MOSFET gates and RTC lines.

What each main part means

Part / label	Plain meaning	What to check
12VINPUT / KH-GX20-2P	Connector where the 12V DC supply enters.	Pin 1 should be +12V and pin 2 should be GND, if that is how the real connector is wired.
Arduino Nano ESP32	The controller or brain of the system.	Power it correctly. Do not send 12V into normal ESP32 pins. Most ESP32 logic pins are 3.3V.
RTC module	A clock that lets the Arduino know the time.	Connect VCC to the correct voltage, GND to ground, SDA to SDA, and SCL to SCL. SQW is optional.
Blue Driver 700 mA	Current driver for the blue 440 nm 3W LED.	Confirm IN, GND, VOUT, and FEED wiring with the driver datasheet.
Red Driver 700 mA	Current driver for the red 660 nm 3W LED.	Confirm driver output polarity and whether the MOSFET can be used on the low side.
D1 / D2 LEDs	High-power blue and red LEDs.	Make sure LED polarity is correct: anode to positive driver output, cathode toward switching path.
Q1 / Q2 MOSFETs	Electronic switches for the LED channels.	Use logic-level N-channel MOSFETs. Source should go to GND for low-side switching.
220 ohm gate resistors	Small resistors between Arduino output pins and MOSFET gates.	Each MOSFET gate should have one series resistor from the Arduino control pin.
10k pulldown resistors	Keep MOSFET gates off when Arduino is not actively driving them.	Each MOSFET gate should have a 10k resistor to GND, not to +12V.
Common ground	Shared ground reference for all low-voltage control and power sections.	All GND points must be connected together: Arduino, RTC, drivers, MOSFET sources, and power supply GND.

Filled Last Section - User Stories Supported by the Prototype

This section connects the schematic to the iGEM cultivator user stories in simple language. It explains what the Arduino, RTC, Wi-Fi, MOSFETs, and LEDs are expected to do in the overall system.

What this schematic already supports

Timed light control, red/blue LED switching, MOSFET-based dimming/control, shared ground, RTC timekeeping, and controller-based automation.

What may need extra hardware later

pH, nutrient, CO₂, biomass, aeration/turbidity, temperature safety sensors, harvesting mechanism, and full app/database integration.

Filled user story table

Role	Priority	User story / action	Simple outcome	Schematic support
Arduino	Must	Use the RTC to keep track of date and time for the cultivator schedule.	The system can run light cycles automatically instead of relying on manual timing.	RTC connects to the Arduino using SDA/SCL, with VCC and GND.
Arduino	Must	Turn the red and blue LED channels on/off based on the programmed schedule.	The cultivator can give duckweed the correct light exposure at the correct time.	Arduino output pins control Q1 and Q2 MOSFET switches.
Arduino	Should	Dim or brighten the LED channels independently.	Light intensity can be adjusted for different growth settings.	Use PWM-capable Arduino pins through 220 ohm gate resistors into logic-level MOSFET gates.
Arduino	Must	Turn off the machine or lights if the system gets too hot.	The prototype is safer and reduces overheating risk.	Add a temperature sensor and code-based shutdown rule. Current schematic shows the control path but not the sensor yet.
Arduino / App	Should	Send light status and sensor readings to the app or database through Wi-Fi/API calls.	The dashboard can show current cultivator status and history.	Arduino Nano ESP32 has Wi-Fi capability; app/database connection is a software feature.
User	Must	Select a pre-created crop or fertilizer setting and fine-tune it manually.	A farmer can start from a template and adjust settings for their crop or region.	The Arduino can apply the selected settings to LED timing/intensity and later to sensors/actuators.
User	Should	Receive alerts when parameters move outside the ideal range.	The app can warn the user before crop yield is affected.	Requires sensors for pH, nutrients, CO ₂ , water temperature, biomass, and aeration.
Engineer	Should	Automatically restart or safely reset the controller when needed for stable 24/7 running.	The cultivator behaves more reliably during long experiments.	This is mainly a software/watchdog function; hardware reset control can be added if required.
User	Could	View historical performance for a saved setting.	Users can compare recipes and improve future growth cycles.	Requires data logging from the Arduino/app and a database or saved file.

Simple final explanation: The prototype is mainly the hardware foundation for timed light control. The Arduino reads time from the RTC, uses MOSFETs to control the red and blue LED channels, and can later send data to the app. Sensor-based alerts, performance history, and automatic harvesting are future extensions that need additional sensors, software, and possibly motors/actuators.